

FINAL PRESENTATION

SE1 INTERNSHIP 2026

**STATIC ANALYSIS WITH WALA:
A DEAD-STORE DETECTOR FOR JAVA**

PRESENTED BY
Prawit Pongpipat

UNDER THE SUPERVISION OF
Prof. Dr.-Ing. Christian Hammer
Dr. Mohammad Rezaalipour
Prof. Somchoke Ruengittinun



WHAT IS STATIC ANALYSIS?

- **Static Analysis = The study of a program's behavior without executing the program**
- A static analyzer reasons about every possible execution directly from the source code or compiled form
- Where it's foundational: *fault localization, security vulnerability detection, code-clone identification, automated refactoring, compiler optimization, etc.*
- The everyday version → **warnings like "this variable is never used" or "unreachable code"** in modern IDEs and linters are static analysis running quietly in the background

Static Code Analysis	VS	Dynamic Code Analysis
Analyzes code without executing	Definition	Analyzes code during execution
During development, before runtime	When Performed	During testing or production
Code quality, standards compliance, and security	Focus	Performance, runtime errors, and memory leaks
Static analyzers (e.g., SonarQube, custom linters)	Tools Used	Debuggers, profilers, runtime monitoring tools

```
int helperField = 0; no usages
```

WHAT IS DEAD CODE/DEAD STORE?

- **Dead code** = Code that doesn't affect a program's observable results
- **Dead store** = An assignment to a variable whose value is never read before it's overwritten or the program ends
- Why it's worth detecting: dead stores rarely appear on purpose — they accumulate through refactoring that leaves an assignment behind. Detecting them keeps the code clean and can flag a genuine logic error: a result computed but accidentally never used.

```
public class Main {  
    public static void main(String[] args) {  
        int active = 10;  
        int dead = 20;  
        System.out.println(active);  
    }  
}
```

“active” is read by println → live

“dead” is assigned but never used anywhere → dead store

WHAT IS WALA?

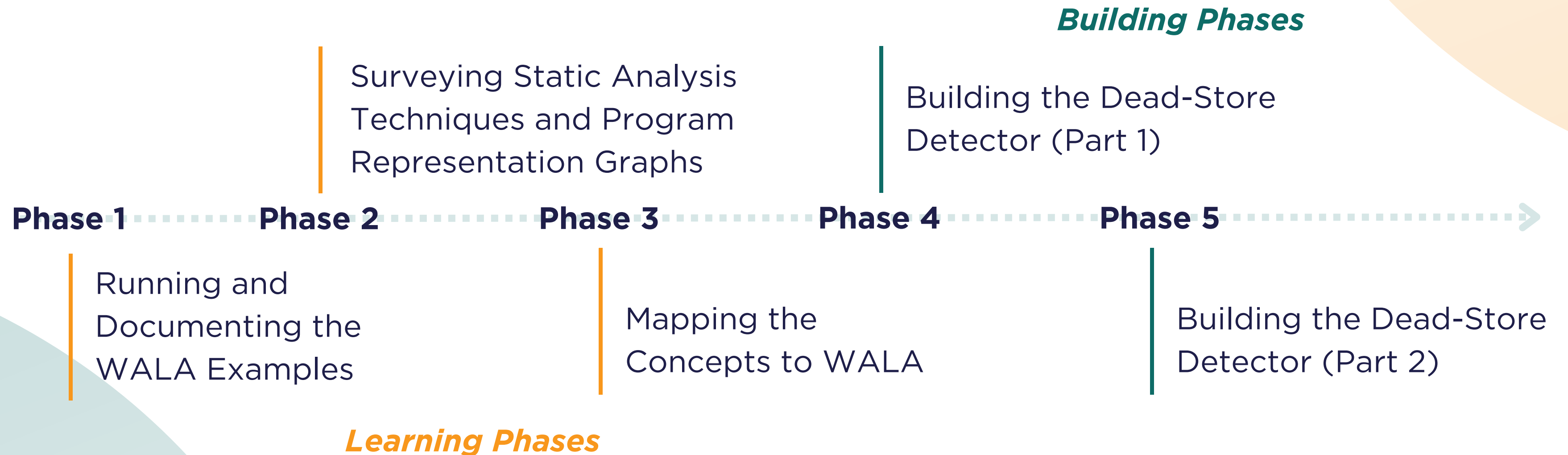
- **Open-source framework for static analysis of Java bytecode** (and JavaScript); infrastructure originally built as part of IBM's DOMO research project, open-sourced in 2006 under the Eclipse Public License
- **WALA is not a single analysis tool but a framework** → Reusable building blocks that specific analyses are constructed on top of.
- It deliberately stops short of being a compiler: **no code-generation back end, and its IR is designed for reading and reasoning**, not transforming.
- Core capabilities → Class hierarchy analysis, call graph construction + pointer analysis, an SSA-form intermediate representation.
- Real-world use → Taint analysis in IBM's Rational AppScan, and null-pointer and resource-leak detection in Rational Software Analyzer
- As of 2026, official WALA releases target Java 17+ and use Gradle as the build system



WALA

T. J. WATSON LIBRARIES FOR ANALYSIS

MY ROADMAP AT A GLANCE



PHASE 1

- **Get every WALA-start example running, documented well enough for someone else to reproduce**
- Environment was the hard part: Gradle+Spotless formatting, JDK 9+ dropping rt.jar (had to rebuild java.base.jar), a missing exclusions file
- **7 example drivers run** → covering class hierarchy, call graphs, pointer analysis, reaching definitions, SSA IR
- JDK compatibility: 8/11 fail (Gradle needs 17+), 17/25 pass → **JDK 17 became the baseline**
- Reproducibility check across the team caught small gaps in run instructions

Driver	Function
PrintTypeHierarchy	Prints the class hierarchy of an application as text
PDFTTypeHierarchy	Renders the class hierarchy as a visual PDF diagram via Graphviz
ScopeFileCallGraph	Builds a call graph from compiled bytecode, using a scope file
SourceDirCallGraph	Builds a call graph directly from Java source
CSReachingDefsDriver	Performs interprocedural, context-sensitive reaching-definitions analysis
DemandPointsToDriver	Performs demand-driven points-to analysis for queried variables
ConstructAllIRs	Constructs the SSA intermediate representation for every method

JDK version	Result	Notes
JDK 8	Fail	Gradle requires JVM 17 or later; build refused to run
JDK 11	Fail	Gradle requires JVM 17 or later; build refused to run
JDK 17	Pass	All drivers ran successfully
JDK 25	Pass	All drivers ran successfully

PHASE 1

<https://github.com/wala/WALA-start>

WALA-startPublic

Watch4Fork31Star29

master1 Branch0 Tags

Go to fileTAdd fileCode

dependabot[bot] Bump com.diffplug.spotless from 8.8.0 to 8.9.0 (#42) b89aa98 · last week 95 Commits

.github	Update and refine GitHub actions (#27)	4 months ago
gradle/wrapper	Bump gradle-wrapper from 9.6.0 to 9.6.1 (#39)	2 months ago
src/main/java/com/ibm/wala/examples	Upgrade to WALA 1.7.1 (#30)	4 months ago
.gitignore	update ignores	10 years ago
LICENSE.txt	docs and license file	9 years ago
README.md	update README	2 years ago
build.gradle.kts	Bump com.diffplug.spotless from 8.8.0 to 8.9.0 (#42)	last week
gradlew	Bump gradle-wrapper from 9.5.1 to 9.6.0 (#38)	2 months ago
gradlew.bat	Bump gradle-wrapper from 9.5.1 to 9.6.0 (#38)	2 months ago
run.py	add an example and make running apps easier	9 years ago
settings.gradle.kts	Convert Gradle build logic from Groovy to Kotlin (#26)	4 months ago

READMEEPL-1.0 license

WALA Starter Kit

About

Basic Gradle configuration and example drivers to get started with WALA

ReadmeEPL-1.0 licenseActivityCustom properties29 stars4 watching31 forksReport repository

Contributors6

Languages

Java 99.5%Python 0.5%

PHASE 1

PDFTypeHierarchy

Table of Contents

- Functionality
- Input & Output
- Run Instructions
 - Prerequisites
 - Steps
 - Sample Input
 - Expected Output
- Reproducibility

Functionality

PDFTypeHierarchy builds a class hierarchy from compiled Java bytecode and renders it as a visual diagram as a PDF file. It uses Graphviz (dot) to convert the hierarchy into a directed graph, where edges point from parent classes/interfaces to their subtypes, then opens the result in the system's default PDF viewer. Only classes loaded by the Application class loader are shown (JDK internals are pruned out.)

Input & Output

- **Input:** A classpath pointing to a directory of compiled .class file or .jar file, via the -classpath flag.
- **Output:** A PDF file (written to a system temporary directory) that opens automatically, showing each application class as a node with directed edges from parent to its subtype.

Run Instructions

Prerequisites

1. **Java version 21** — verify with java -version
2. **Graphviz** — required to render the graph to PDF. Install on macOS with:

```
brew install graphviz
```

Verify with:

```
which dot
```

```
public class AnalysisClass {  
  
    public static long up(int n) {  
        if (n <= 1) return n;  
        long[] dp = new long[n + 1];  
        dp[0] = 0; dp[1] = 1;  
        for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];  
        return dp[n];  
    }  
  
    private static long[] memo = new long[100];  
  
    public static long down(int n) {  
        if (n <= 1) return n;  
        if (memo[n] != 0) return memo[n];  
        memo[n] = down(n - 1) + down(n - 2);  
        return memo[n];  
    }  
  
    public static void run() {  
        int n = 10;  
        System.out.println("Fibonacci DP Demo (n = " + n + ")");  
        System.out.println("Bottom-up : " + up(n));  
        System.out.println("Top-down   : " + down(n));  
        System.out.println("\nSequence (0.." + n + "):");  
        for (int i = 0; i <= n; i++) {  
            System.out.print(up(i) + (i < n ? " " : "\n"));  
        }  
    }  
}
```

Expected Output

A PDF diagram opens showing the inheritance chain for application classes only:

```
graph TD; Root["<Primordial,Ljava/lang/Object>"] --> Left["<Application, Lcom/sirisuk/AnalysisClass>"]; Root --> Right["<Application, Lcom/sirisuk/Main>"];
```

Both Main and AnalysisClass have no explicit superclass, so they both inherit directly from java.lang.Object — the root of all Java classes. The diagram has exactly 3 nodes and 2 edges.

Note: The PDF is written to a temporary file (e.g. /tmp/out1228629807.pdf) and opened automatically. The exact filename changes on every run.

Reproducibility

No reproduce section on this driver, since this was created by myself, Prawit.

PHASE 2

- **Catalogued 10 static-analysis techniques + 10 program representation graphs**, each backed by an academic reference
- Breadth-first → this built a shared vocabulary before any code was written
- **Live-variable analysis is the conceptual root of a dead store**

Technique	What it determines
Constant Propagation	The compile-time value of a variable when it never changes
Sign Analysis	Whether a variable is positive, negative, or zero
Reaching Definitions	Which assignments might have last set a variable's value
Live Variable Analysis	Which variables might still be read in the future
Available Expressions	Which already-computed expressions are still valid
Taint Analysis	Whether untrusted data can reach a sensitive operation
Points-to Analysis	Which objects a reference might point to
Type Inference	A variable's type, inferred from how it is used
Null Pointer Analysis	Whether a variable might be null when dereferenced
Interval Analysis	The range of values a variable can hold

PHASE 3

- **Central finding → WALA is a framework, not a toolbox**
- Ready-made: class hierarchy, call graph, PDG/SDG/CDG, SSA IR, points-to analysis
- Build-it-yourself: classical dataflow analyses (constant propagation, live variables, taint) sit on WALA's general dataflow framework instead
- This distinction is what made Phase 4 and 5 buildable

Concept	WALA implementation
Control Flow Graph	ControlFlowGraph / SSACFG
Call Graph	CallGraph / BasicCallGraph
Program Dependence Graph	PDG
System Dependence Graph	SDG
Control Dependence Graph	ControlDependenceGraph
SSA Form	IR / SSACFG
Interprocedural CFG	InterproceduralCFG
Heap Graph	HeapGraph / BasicHeapGraph
Points-to Analysis	PointerAnalysis / DemandRefinementPointsTo
Type Inference	TypeInference
Reaching Definitions	via the IFDS TabulationSolver

Concept	How WALA supports it
Constant Propagation	Built on the dataflow framework; constants tracked via SymbolTable
Sign Analysis	Built on the dataflow framework
Live Variable Analysis	Built on the dataflow framework (backward)
Available Expressions	Built on the dataflow framework
Taint Analysis	Built on the IFDS solver over call graph + pointer analysis

PHASE 4

- **First real build: a dead-store detector on top of WALA, run from the command line on a single .java file**
- Pipeline in one breath: **compile with -g → scope to app classes → build class hierarchy → SSA IR + DefUse → recover name & line → report**
- Key trick: iterate over every value, not instructions → `int dead = 20` emits no store instruction at all
- **Interprocedural cascade:** a value used only to feed a parameter its callee never reads is still effectively dead

Test	Scenario	Result
test1	A basic dead store (<code>int dead = 20</code>)	Reports “dead”
test2	All variables are used	No dead stores
test3	A variable overwritten before it is read	Reports the first assignment of the variable
test4	Branches, a loop, three methods, and the cascade	Reports four dead stores, incl. the cascade
test5	Two locals holding the same literal	Known limitation in DefUse

PHASE 5

- Chose to deepen the dead-store detector rather than start the team's CFG project from scratch — more productive use of remaining time
- **Scope: nested classes, multiple methods; broadened definition to locals + fields + parameters, and multiple files/directories**
- **Removed the interprocedural cascade** — deliberate, spec-driven decision
- **Same test4 input, different correct answer** → test is now LIVE, not dead
- Three mechanisms: DefUse for locals, no-uses-in-body for parameters, write-without-read across methods for fields
- Verified working: nested classes to any depth, synthetic parameters correctly suppressed

Test	Scenario	Result
mohammad	The specification's own example	Reports all three categories
test1–test3	Basic local dead stores (carried over from Phase 4)	Reports locals, cascade-free
test4	Branches, loop, unused field and parameter	Reports field, parameter, and locals
test5	Two locals holding the same literal	Known limitation in DefUse
test6	A class nested inside a nested class	Reports fields, parameters, and a local across all levels
test7	A used field alongside an unused one	Reports only the unused field
test8	A static nested class	Reports field, parameter, and local
test9	Two files in one directory (cross-file field use)	Correctly sees the cross-file read
test10	Two packages in separate directories	Correctly sees the cross-package read

LIMITATIONS

- **Two locals, same literal → missed dead store**
 - WALA interns constants: `int a = 5; int b = 5;` share one value number
 - Reading one hides the other; `b` never flagged
 - Fix needs bytecode-level analysis (Shrike) — out of scope; kept as test5
- **Parameter line numbers → off by 1-2 lines**
 - Compiler omits the signature line from the class file; can't recover it
 - Still points to the correct method
- **Local line numbers → rely on a compiler-layout assumption**
 - Held on every test, but not guaranteed by WALA's API



Tested and confirmed to work:

- Nested classes — any depth
- Synthetic params (`this$0`) and main's args — suppressed
- Fields across methods — cross-method reads seen
- Multiple files & packages — cross-file / cross-package reads seen (test9, test10)



TESTING

mohammad.java → Dr. Mohammad Rezaalipour's test to my dead-store detector.

```
1 package com.example;
2
3 public class Main {
4     private class MyNestedClass {
5         public int myNestedField = 12; // DEAD STORE
6
7         private void myNestedMethod(int myPar1) { // DEAD STORE
8             System.out.println("Method in the nested class.");
9         }
10    }
11
12    private static int myField = 13; // DEAD STORE
13    public static void main(String[] args) {
14        int myVar = 14;
15        int myVar2 = 23; // DEAD STORE
16
17        myMethod(myVar);
18    }
19
20    private static void myMethod(int myPar2) { // DEAD STORE
21        System.out.println("Method in the class.");
22    }
23 }
```

Terminal

Local × + ∨

```
prawit@Prawits-MacBook-Pro phase_05 % ./gradlew -q run --args="tests/mohammad"
Dead store detected:
  Field: myNestedField, Line: 5 at "com/example/Main.java"
  Field: myField, Line: 12 at "com/example/Main.java"
  Parameter: myPar1, Line: 8 at "com/example/Main.java"
  Parameter: myPar2, Line: 21 at "com/example/Main.java"
  Variable: myVar2, Line: 15 at "com/example/Main.java"

prawit@Prawits-MacBook-Pro phase_05 %
```

WHAT I HAVE LEARNED

Read a specification carefully

Phase 4 and 5 quietly wanted different behavior for the same case

Understand a tool before building on it

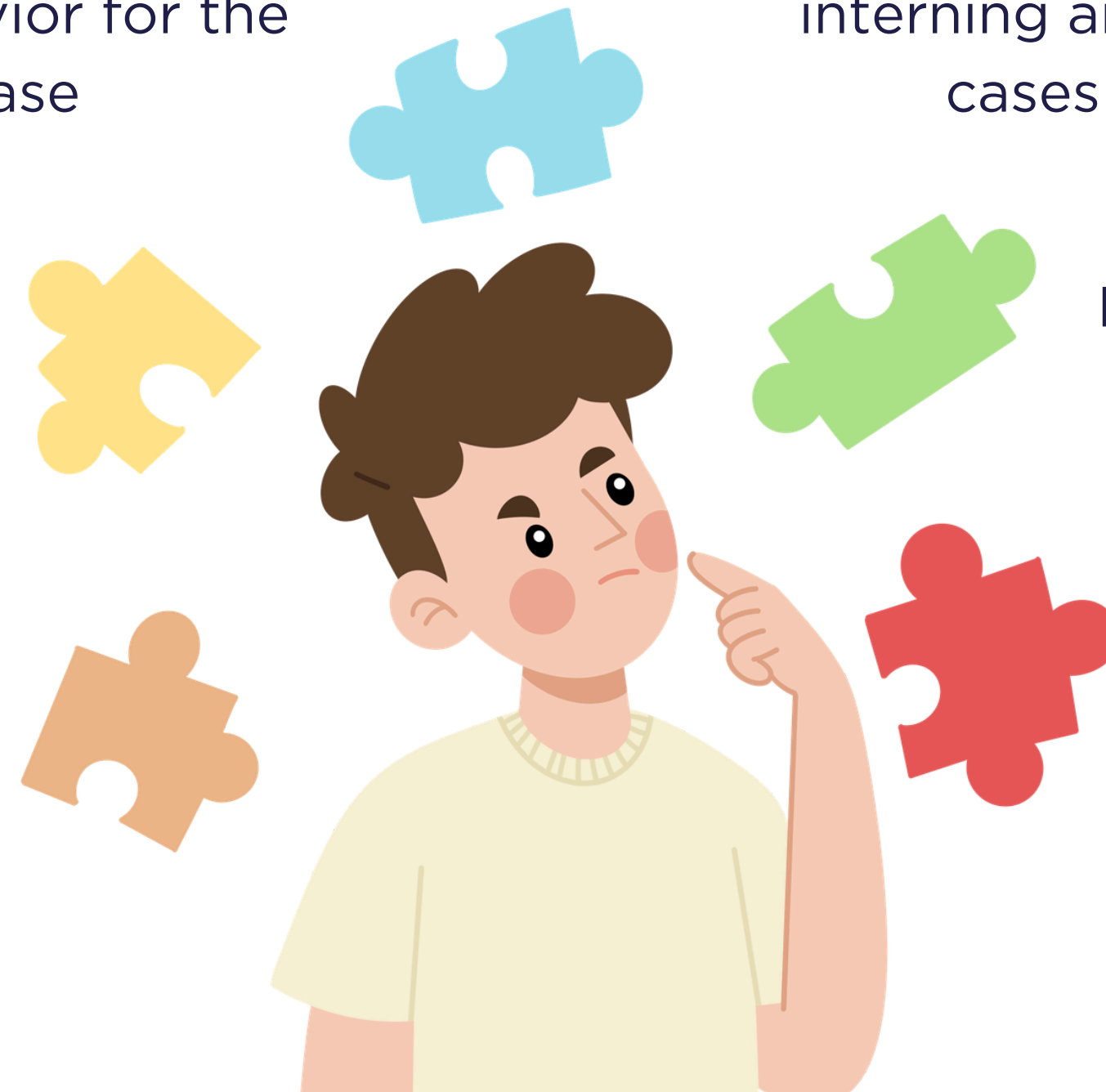
The slow phase 1 to 3 made the later builds possible

Debug by looking at WALA's IR directly, not guessing

That's how the constant-interning and no-instruction cases were found

Document limits honestly

A known failure case, recorded and explained, beats one left out

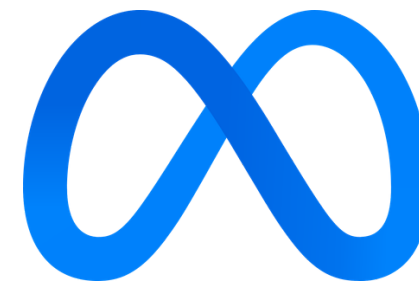


FUTURE DIRECTIONS

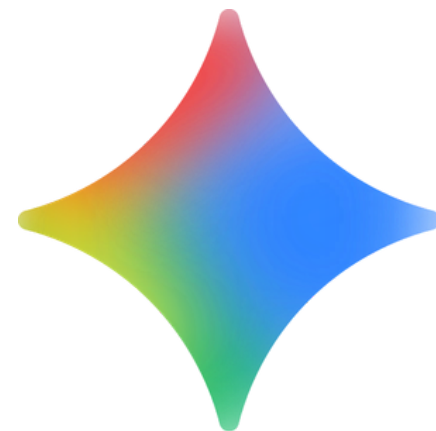
- **Apply static analysis to the security of AI-generated source code**
 - Generate solutions from several different LLMs
 - Run static security analyses over each
 - Compare which vulnerabilities each model tends to introduce
- Builds directly on this internship; connects to the current trends of AI
- "My last role (this project) was about code cleanup. This one is about code security."



GPT Series
Reasoning Series (o-series)



LLAMA Series



Flash
Pro



Sonnet
Opus
Fable



Deepseek Series

And many more...

FINAL PRESENTATION

SE1 INTERNSHIP 2026

THANK YOU FOR LISTENING!

PRESENTED BY
Prawit Pongpipat

